IT5437 — Computer Vision

Assignment 2: Fitting and Alignment

Benjamine S. | Registration No: 258762A

GitHub: https://github.com/ITToday/IT5437-Assignment-2

1(a) Total Least Squares Line Fitting

Total Least Squares (TLS) was used to fit a line to the first point scatter (x1, y1) from lines.csv. Unlike ordinary least squares which only minimises vertical errors, TLS minimises the **perpendicular distance** from each point to the line. This is done using SVD on the mean-centred data matrix. The last row of V^T (smallest singular value) gives the normal vector [a, b] of the line.

Key code:

```
# mean-centre the data
A = np.column_stack((x1 - x_mean, y1 - y_mean))

# apply SVD - normal vector = last row of Vt (smallest singular value)
U, S, Vt = np.linalg.svd(A)
a, b = Vt[-1]
d = a * x_mean + b * y_mean

# convert to slope-intercept for plotting
m = -a / b
c_intercept = d / b
```

Results: Normal vector [a, b] = [0.7736, –0.6337] | Offset d = 3.7942 | Line equation: 0.7736x – 0.6337y = 3.7942 | Slope m = 1.2207, Intercept c = –5.9872

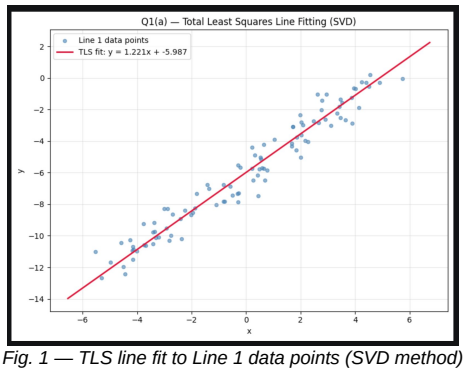

Fig. 1 — TLS line fit to Line 1 data points (SVD method)

1(b) RANSAC Line Fitting for Three Lines

All points (x1, x2, x3 and y1, y2, y3) were flattened into one array as given in the assignment code snippet. RANSAC was run three times — each run finds one line, removes its inliers, then the next run works on the remaining points. For each iteration, 2 points are sampled randomly and a line is fitted using TLS/SVD. Perpendicular distance is computed for all points and those within threshold=0.5 are inliers. After 1000 iterations, all inliers are used to refit the final line.

Key code:

```
for _ in range(n_iterations):
  idx = np.random.choice(len(X), size=2)
  # fit TLS line to 2 sampled points
  distances = np.abs(a*X + b*Y - d) / np.sqrt(a**2 + b**2)
  inlier_mask = distances < threshold
```

Line	Equation (ax+by=d)	Slope m	Intercept c	Inliers
1	0.4512x + 0.8924y = 1.9458	–0.506	2.180	83
2	0.7353x – 0.6778y = –0.8900	1.085	1.313	67
3	–0.7928x + 0.6095y = –3.6402	1.301	–5.973	64

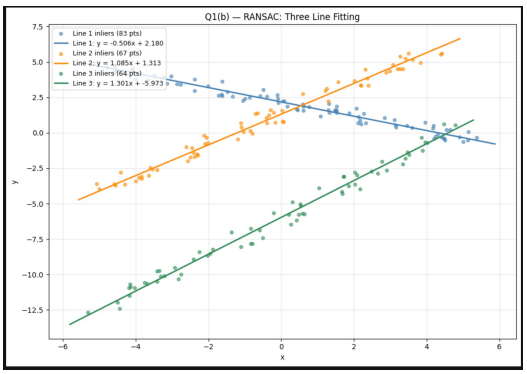

Fig. 2 — RANSAC: three lines fitted to all 300 combined points (86 unassigned)

RANSAC correctly separated all three clusters. 86 points were not assigned to any line — these are genuine outliers or points between clusters. The result confirms that iterative inlier removal works well when clusters do not heavily overlap.

2 Earring Size Estimation — Pinhole Camera Model

Given: focal length f = 8 mm, pixel size = 2.2 µm = 0.0022 mm, distance from lens to imaging plane Z = 720 mm. The pinhole camera model gives the real-world size from the image size through similar triangles:

```
real_size = (pixel_count × pixel_size × Z) / f
```

The earring was isolated using binary thresholding (pixels darker than 200 become white). cv2.findContours found the outer boundary and cv2.minEnclosingCircle gave the outer radius. The inner hole radius was measured by scanning horizontally through the earring centre to find where the ring material starts on each side.

Fig. 3 — Left: detected outer contours (red). Right: outer (red) and inner hole (green) measurements.

Measurement	Pixels	Real Size (mm)
Outer diameter	403.2 px	79.82 mm
Inner diameter	250.0 px	49.50 mm
Ring thickness	76.6 px	15.16 mm

The outer diameter of ~79.8 mm is realistic for a large hoop earring. Accuracy depends on how precisely the pixel measurements can be made from the image.

3 Circuit Board Homography and Feature Matching

3(a) Manual Homography and Warp

Since cv2.setMouseCallback does not work in Jupyter notebooks, 6 corresponding landmark points were visually identified from a grid display and recorded manually. Landmarks: DC power jack, USB port corner, two mounting holes, and two corners of the ATmega2560 chip. cv2.findHomography (with RANSAC) computed the 3×3 homography matrix H. cv2.warpPerspective then mapped c1 to the perspective of c2.

Key code:

```
H, mask = cv2.findHomography(pts1, pts2, cv2.RANSAC, 5.0)
im1_warped = cv2.warpPerspective(im1, H, (w, h))
```

Homography H: RANSAC used 5 of 6 points as inliers (one point was slightly off the board).

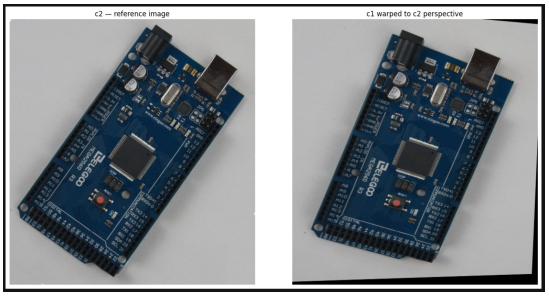

Fig. 4 — Left: c2 reference. Right: c1 warped to c2 perspective using manual homography.

3(b) Difference Image — Manual Homography

cv2.absdiff was used to compute the absolute pixel difference between c2 and the warped c1. The result was enhanced ×3 to make small differences visible. Mean difference = 32.18, pixels with diff > 30 = 286,577.

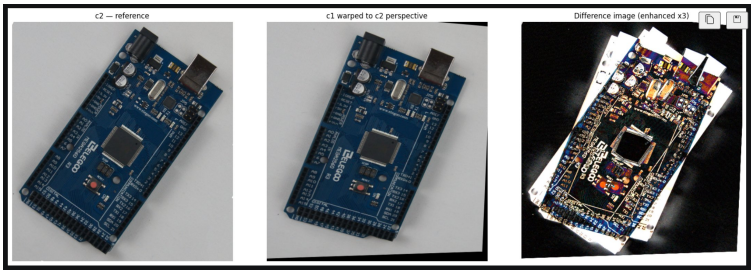

Fig. 5 — Difference image (manual): two overlapping board outlines visible due to imperfect manual alignment.

3(c) ORB Feature Matching

ORB (Oriented FAST and Rotated BRIEF) was used to detect 500 keypoints per image and compute 256-bit binary descriptors. BFMatcher with cv2.NORM_HAMMING and crossCheck=True matched descriptors in both directions to reduce false matches. 363 matches were found. Best match Hamming distance = 8 (very low = high confidence).

Key code:

```
orb = cv2.ORB_create(nfeatures=500)
kp1, des1 = orb.detectAndCompute(im1, None)
bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
matches = sorted(bf.match(des1, des2), key=lambda x: x.distance)
```

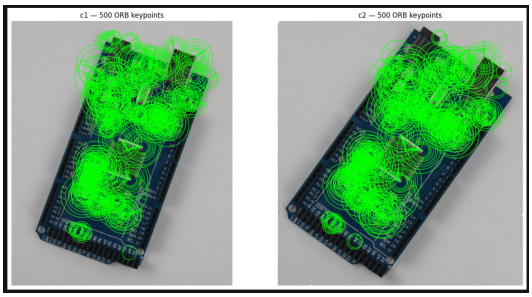

Fig. 6 — Top 50 ORB matches (green lines) out of 363 total. Matches connect correct board regions.

3(d) ORB Homography, Difference Image and Comparison

Matched keypoint coordinates were extracted and passed to cv2.findHomography. RANSAC accepted 359 of 363 matches (98.9% inlier rate). The resulting warp and difference image were compared against the manual approach.

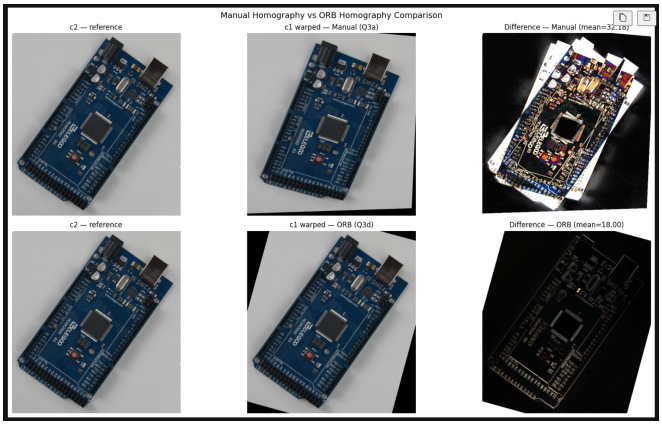

Fig. 7 — Top row: manual method. Bottom row: ORB method. Mean difference reduced from 32.18 to 18.00.

Metric	Manual (Q3a/b)	ORB (Q3d)
Point pairs used	6	363
RANSAC inliers	5 (83%)	359 (98.9%)
Mean pixel difference	32.18	18.00
Pixels with diff > 30	286,577	104,531

Discussion:

ORB gave a significantly better result — mean difference dropped by 44% and large-error pixels reduced by 63%. This is because ORB used 363 precise correspondences spread across the full board, while manual selection only had 6 visually estimated points with human error of several pixels each. ORB also achieved a 98.9% RANSAC inlier rate vs 83% for manual, showing the matches were much more

geometrically consistent.

Even with ORB, the difference is not zero because: (1) the two photos have slightly different lighting which no geometric transform can correct; (2) cv2.warpPerspective fills unmapped border pixels with black creating artificial differences at image edges; (3) small residual misalignment remains from the 4 rejected RANSAC outliers.